

Análisis organizado de TallerRNA.ipynb

Archivo: Laboratorios/Taller 3/TallerRNA.ipynb

Propósito general

- El cuaderno implementa un experimento completo de clasificación facial con una red neuronal densa tipo MLP sobre el dataset Olivetti Faces.
- El objetivo es entrenar varias variantes del modelo, compararlas con validación y conservar las mejores para evaluarlas después en prueba.
- El flujo general va de carga de datos a partición estratificada, normalización, búsqueda aleatoria de hiperparámetros, selección top-3 y análisis final en test.

Carga y partición de datos

- El dataset se carga con `fetch_olivetti_faces` y se trabaja con 400 imágenes aplanadas de tamaño 64 x 64, lo que produce vectores de 4096 características por imagen.
- La división se hace en dos pasos con `train_test_split` y estratificación: primero 60% para entrenamiento y 40% para temporal, y luego el temporal se divide en partes iguales para validación y prueba.
- Con esa estrategia se obtiene aproximadamente una distribución de 6 imágenes por identidad para entrenamiento, 2 para validación y 2 para prueba, manteniendo representación equilibrada en los tres subconjuntos.

Normalización y reproducibilidad

- La normalización se realiza con `StandardScaler` ajustado únicamente sobre entrenamiento, y luego aplicado a validación y prueba con los mismos parámetros.
- Esto evita fuga de información desde los conjuntos de evaluación y ayuda a estabilizar el entrenamiento en un espacio de 4096 variables.
- También se fijan semillas aleatorias para NumPy, TensorFlow y random, con el fin de mejorar la reproducibilidad de los resultados.

Arquitecturas del MLP

- La función `build_model` construye tres perfiles: Robusto, Profundo y Adaptativo.
- El perfil Robusto usa Dense con ReLU, HeNormal, Dropout y optimizador Adam.
- El perfil Profundo usa Dense con SELU, LecunNormal, regularización L2 y AdamW, con fallback a Adam si la versión de TensorFlow no soporta AdamW.

- El perfil Adaptativo usa Dense con LeakyReLU, HeNormal y RMSprop, además de ReduceLROnPlateau para bajar la tasa de aprendizaje cuando la validación se estanca.
- Todas las variantes terminan en una capa softmax de 40 neuronas, una por cada clase del problema multiclase.

Búsqueda aleatoria

- El cuaderno ejecuta una búsqueda aleatoria de 50 iteraciones.
- En cada iteración se elige al azar un perfil, una tasa de aprendizaje y varios hiperparámetros dependientes de la arquitectura.
- El entrenamiento puede llegar hasta 150 épocas y usa EarlyStopping para detenerse cuando la validación deja de mejorar.
- Para el perfil Adaptativo se agrega además ReduceLROnPlateau, y al final se guarda la mejor validación, el historial de entrenamiento y los pesos del mejor punto encontrado.

Selección del top 3

- Con los resultados de la búsqueda, el cuaderno ordena los modelos por val_accuracy y conserva los tres mejores.
- Esa decisión se toma exclusivamente con validación, no con prueba, para evitar sesgo por sobreajuste al conjunto test.
- Por eso el orden del top 3 no necesariamente coincide con el mejor test accuracy.
- En las salidas guardadas, los tres mejores modelos pertenecen al perfil Robusto.

Evaluación de los mejores modelos

- Cada uno de los tres modelos se reconstruye exactamente con sus hiperparámetros y con los pesos guardados.
- Después se grafican las curvas de loss y accuracy de entrenamiento frente a validación para inspeccionar la convergencia.
- También se evalúa el conjunto de prueba, se reporta accuracy, classification report y matriz de confusión.
- Esta etapa permite comparar desempeño cuantitativo y comportamiento de cada arquitectura elegida.

Inspección cualitativa

- El cuaderno incluye una función para mostrar imágenes aleatorias de prueba junto con la predicción y la clase real.
- También incorpora otra función para localizar todos los errores de clasificación y visualizarlos.

- Estas herramientas ayudan a entender qué identidades se confunden y si los fallos tienen sentido visual.

Conclusión técnica

- El comportamiento general sugiere que, para este conjunto pequeño, el perfil Robusto generaliza mejor.
- Las curvas muestran una convergencia rápida y estable, con poca separación entre train y validation, lo que apunta a un control razonable del sobreajuste.
- El resultado final queda cerca de 95% de accuracy en prueba y alrededor de 0.97 de macro F1, con errores concentrados en pocas clases.

Resumen técnico por apartados

Apartado	Qué hace	Idea clave
Partición	Divide 400 imágenes en train/val/test de forma estratificada.	Preserva la representación por identidad.
Normalización	Ajusta StandardScaler solo en entrenamiento.	Evita fuga de información.
Modelos	Prueba Robusto, Profundo y Adaptativo.	Compara tres estilos de MLP.
Búsqueda	Explora 50 combinaciones al azar.	Selecciona por validación.
Evaluación	Reconstruye top-3 y mide test.	Separa selección de evaluación final.
Análisis visual	Muestra aciertos y errores.	Facilita interpretación cualitativa.

Observación sobre el código

Hay dos detalles técnicos mejorables: el argumento `neurons_list=[]` en `build_model` usa una lista mutable por defecto, y en versiones de TensorFlow antiguas conviene verificar el nombre exacto de las métricas en `history.history`.

Este documento reorganiza la explicación para que puedas usarla como informe técnico, resumen de laboratorio o base para una entrega académica.